基于 PYNQ 的视频风格渲染系统

摘要

本作品在基于 AMD Zynq™ 7000 SoC 的 ZYNQ 7020 开发板上实现嵌入式实时视频风格化。OV5640 摄像头采集并输入视频信号后，PL 完成 3×3 彩色中值、Kuwahara 保边平滑、gamma 提亮、饱和度混合及 Bayer 抖动量化；PS 运行 PYNQ，负责加载硬件、初始化 OV5640 及抓帧。按键支持处理与直通对照，LED2 指示直通状态。

Kuwahara 核采用 7×7 窗口和四个 4×4 象限，以 Q8 整数判别式代替浮点方差，结合偶数舍入求均值。行缓存、滑动累加和输出保持构成可反压流水线，连续推进时达到 1 像素每时钟。800×480 RTL 仿真输出 384000 像素，数据与行帧标志检查通过；整数模型与浮点参考在去除三像素边界后逐位一致。

已有板上帧率记录为 40.05 fps。处理核在 100 MHz 下理论峰值为 100 Mpixel/s，窗口及内部流水延迟约 24.32 μs，不含中值、DDR 缓存和显示等待。实现占用 LUT 22864、寄存器 27514、BRAM Tile 22.5 和 DSP 19。整机仍有负建立时间裕量。

第一部分　作品概述

1.1 功能与特性

连接开发板的 OV5640 摄像头持续采集现场画面，以 800×480 RGB565 经 DVP 输入；PL 转换为 RGB888 后完成中值、保边平滑和色彩渲染，以 640×480 模式显示。直通同时绕过中值、Kuwahara、gamma、饱和度及 Bayer 量化，保留视频框架和 DDR 缓存。处理支路持续运行以便切回，切换瞬间仍可能跳帧或撕裂。默认 gamma 为 0.45、饱和度系数 1.0、每通道量化 16 级。

1.2 应用领域

作品可用于实时艺术滤镜、边缘设备的视频预处理及 FPGA 图像处理教学。标准流接口便于替换算子；用于机器人或工业视觉时，应结合任务另行评估处理效果。

1.3 主要技术特点

本设计通过四象限选区控制平滑范围，固定整数判别、平局次序和偶数舍入，使测试图的 300516 个内部像素与参考逐位一致。六级环形行缓存与滑动累加复用邻域数据，扩展网格负责补边和排空；显示端以 8×8 Bayer 抖动缓解量化色带。按键同时控制采集与显示两处旁路，便于对照处理和直通效果，切换瞬态另行记录。

1.4 主要性能指标

指标	**结果**
板上帧率	40.05 fps，已有测量记录
处理核吞吐	100 M pixel/s，100 MHz 理论峰值
Kuwahara 核延迟	2432 个推进周期，约 24.32 μs
实现资源	LUT 22864；FF 27514；BRAM 22.5；DSP 19
全尺寸仿真	384000/384000，mismatch=0
建立时间裕量	100 MHz：−0.571 ns；142.857 MHz：−0.115 ns

1.5 主要创新点

1）以整数判别式、确定的平局次序和偶数舍入对齐浮点参考，内部像素实现 0 LSB 差异。

2）将扩展网格与输出保持结合，在密集流及输入间隔测试中保持帧几何，全尺寸仿真 384000 像素无失配。

3）以算法对拍、接口等价仿真和板级观察交叉定位问题：修正 800/640 行宽失配，并验证采集替换模块的消隐行为。创新集中于可复现的硬件实现与集成方法。

1.6 设计流程

以固定参数模型确定 PS 与 PL 分工，在直通基线上集成算法。整数对拍核对数值，RTL 自校验检查流控和边界，板级观察检验真实接口；据重影现象修正行宽，以逐拍仿真确认采集替换行为，形成源码、日志和运行文件对应的提交包。

第二部分　系统组成及功能说明

2.1 整体介绍

系统由摄像头、Zynq 开发板和 HDMI 显示设备构成。PL 的采集处理通路与显示通路通过 VDMA 和 DDR 衔接。PS 不执行逐像素滤波，主要完成启动配置、抓帧及测试控制。图 1 展示最终通路及模式支路。

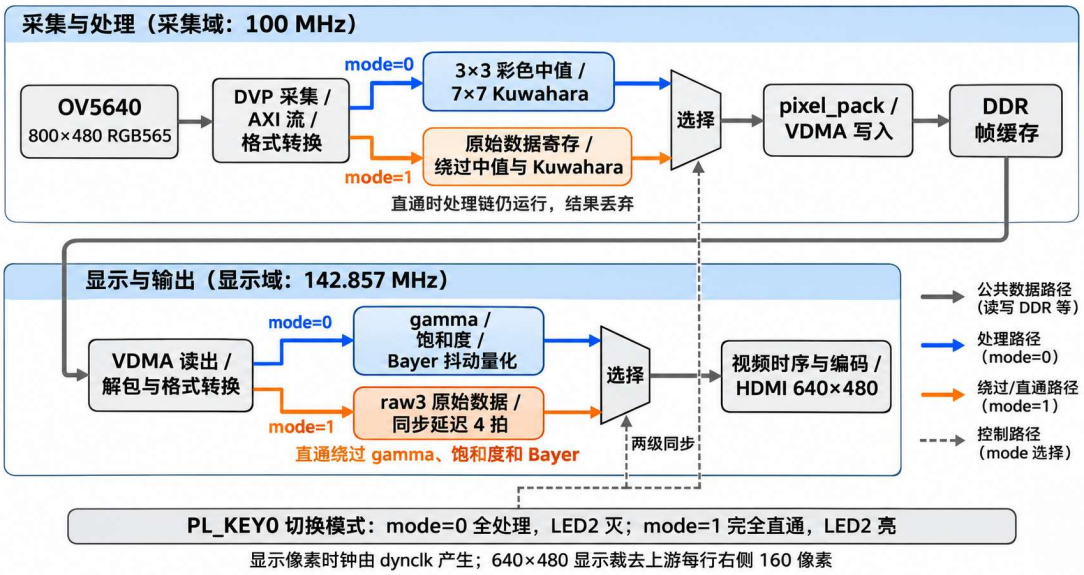


图 1　系统数据通路与控制关系示意图

采集前端将 DVP 数据送入视频转 AXI 流模块，经格式转换与寄存切片后分流。处理模式下，中值与 Kuwahara 的输出进入 pixel_pack 和 VDMA；直通模式下，原始流经寄存后进入相同写帧通路。显示端经 VDMA、pixel_unpack 和 color_convert 后进入 video_style_bypass，最后由视频时序模块与 rgb2dvi 输出。

空间处理的实际几何为 800×480，与上游 HLS 中值 IP 的固定循环一致。当前 PYNQ 显示配置使用 640×480，每行右侧 160 像素不显示。两个尺寸分别约束窗口行缓存和显示搬运。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

实物平台为正点原子领航者 ZYNQ 7020 开发板，底板标识为 ATK-DF7010/7020P；工程目标器件为 xc7z020clg400-2，工具为 Vivado 2025.2。摄像头连接板上 OLED/CAMERA 扩展接口，经 DVP 输入现场画面，PS 经 I2C 初始化 OV5640。采集处理运行于 100 MHz 域，显示点运算运行于 142.857 MHz 域；HDMI 像素时钟由 dynclk 生成。

2.2.2 机械设计介绍

作品采用现有开发板、摄像头与显示设备连接，无独立机械传动或定制外壳。主要设计成果为 PL 逻辑、软硬件接口和控制软件。

2.2.3 电路各模块介绍

模块	输入输出及作用
ov5640_capture	输入 cam_pclk、vsync、href、8 位数据；每两字节拼接 RGB565，低位补零扩展为 RGB888，输出 vid_data、vid_ce 及同步。
采集启动控制	复位同步释放；默认等待前 10 个帧同步计数后，从下一帧起放行；消隐期保持最后一个像素。
median_blur_color	HLS 彩色 3×3 中值模块，对各颜色通道滤波；最终流几何为 800×480。
video_kuwahara	24 位 AXI 流包装与保边平滑核心，生成有效输出对应的 SOF/EOL。
pixel_pack 与 VDMA	像素打包、写入 DDR 与读出显示；引入帧级缓存等待。
video_style_bypass	gamma、Q8 饱和度和 Bayer 量化；支持同步后的运行期旁路。

AXI 视频流使用 TDATA[23:0]、TVALID、TREADY、TUSER[0] 和 TLAST，分别承载像素、握手、帧首与行尾；TKEEP/TSTRB 为三字节有效信息。传输仅在 TVALID 与 TREADY 同时为 1 时发生，阻塞期间数据及边带信号必须保持稳定。[2]

板级接口与摄像头采集设计

板级连接与内部像素处理分层描述如下。实线箭头给出数据方向；cam_pclk 是摄像头输出时钟，I2C 用于寄存器配置，不承载图像像素。DVP 转 AXI 模块之后的视频流统一使用 24 位像素和行帧标志。



图 2　板级连接及关键接口示意

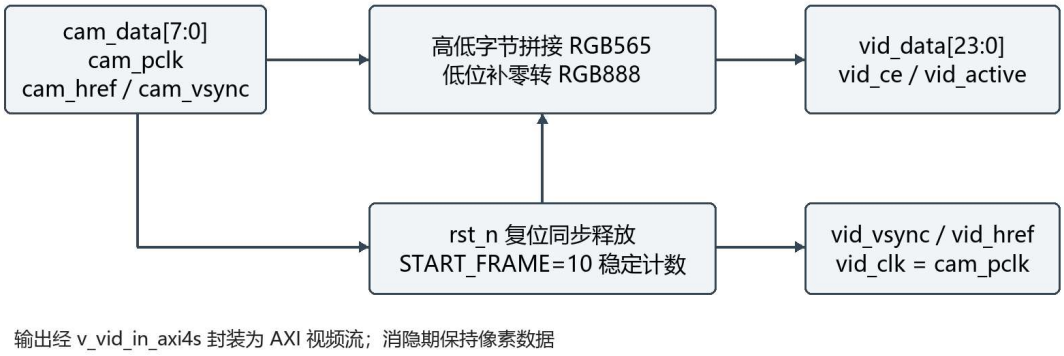


图 3　ov5640_capture 内部设计及关键输入输出

cam_data[7:0] 的两个连续字节组成 RGB565，vid_data[23:0] 由 5 位 R、6 位 G、5 位 B 分别在低位补 3、2、3 个零得到。vid_ce 给出像素使能，vid_active 与有效行对应，vid_vsync/vid_href 为延迟后的同步；rst_n 控制复位，cam_rst_n 固定为 1，cam_pwdn 固定为 0。复位释放和启动帧计数决定何时放行，消隐期保持最后一个像素。

采集流经 color_convert 和寄存切片进入 video_mode_switch。处理支路通过 m_proc 送往 HLS 中值，Kuwahara 输出从 s_proc 返回；选择结果由 m_axis 送 pixel_pack。反压沿各接口的 TREADY 反向传播，不能仅连接数据线。

Kuwahara 核心算法与流水结构

怎样在平滑树叶纹理的同时保留建筑轮廓？Kuwahara 将每个像素的 7×7 邻域分成四个重叠的 4×4 象限，比较亮度方差，再输出最平稳象限的 RGB 均值。跨越物体边缘的象限通常包含较大的亮度变化，因而更可能被避开；这一机制提供保边倾向，但不保证所有细线和纹理都被保留。

设 $Y=77R+150G+29B$，$S=\Sigma Y$，$S_2=\Sigma Y^2$，$N=16$，则判别式 $D=16S_2-S^2$。参考亮度为 $y=Y/256$，因此 $var(y)=D/2^{24}$。正的公共比例因子不影响 argmin。四象限按 TL、TR、BL、BR 顺序用严格小于比较，平局保留先出现的象限。

通道均值用偶数舍入：设通道和 C 的商 $q=C>>4$、余数 $r=C\&15$，则输出 $q+[(r>8)$ 或 $(r=8$ 且 q 为奇数$)]$。RGB 和最大为 4080；Q8 亮度最大为 65280，亮度和小于 2^{20}，D 的计算中间量小于 2^{40}，均可用整数精确表示。

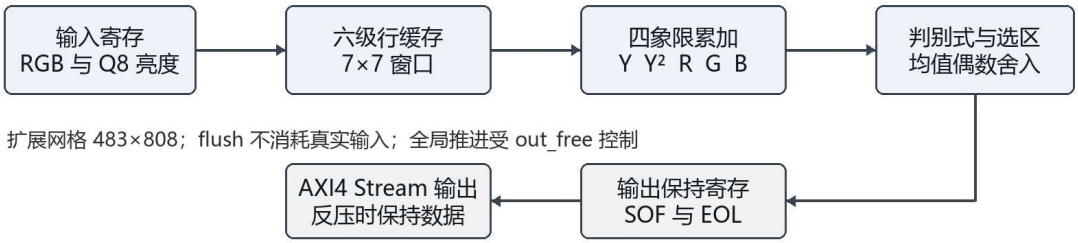


图 4　Kuwahara 数据计算与流控结构

窗口向右移动一个像素时，大部分邻域数据仍可复用。linebuf 用六级环形 BRAM 保存前六行，box4col 累加上下半窗的亮度、平方和与颜色，持续形成四象限统计量。判别式寄存后再比较、选区和求均值，约五级计算及输出保持将长组合运算拆开，支撑连续输入下的单像素吞吐。

扩展网格为 $(H+3)\times(W+8)=483\times808$。真实像素位置消耗输入，补零和排空位置暂停输入握手并自行推进。输出保持寄存器在下游阻塞时暂停流水；输入气泡期间已经被取走的输出清除 valid，避免重复输出。零填充边界与参考的边缘复制策略不同，差异单独统计。

采集路由和显示旁路的局部连接

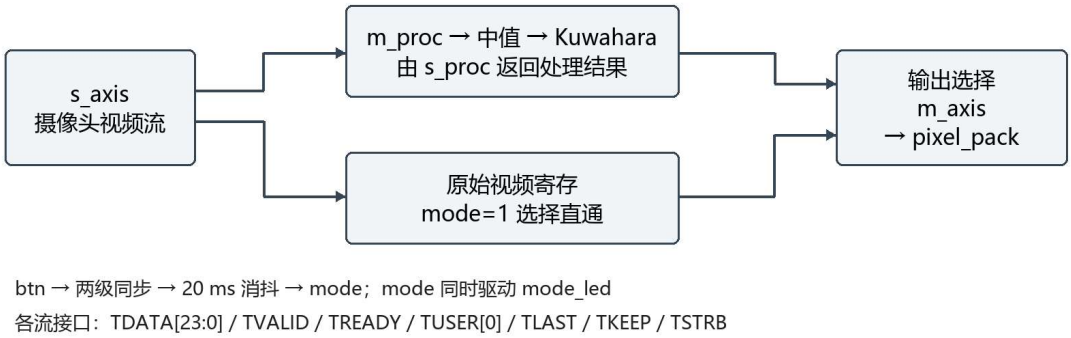


图 5　video_mode_switch 的处理支路与原始支路

模块以 aclk 为时钟、aresetn 为低有效复位。btn 的同步、极性识别和消抖生成 mode，mode_led 指示模式。mode=0 选择 s_proc 结果；mode=1 选择原始支路，同时继续给 m_proc 送数据并丢弃处理输出。各流携带 TVALID/TREADY、TUSER/TLAST 和字节有效标志。

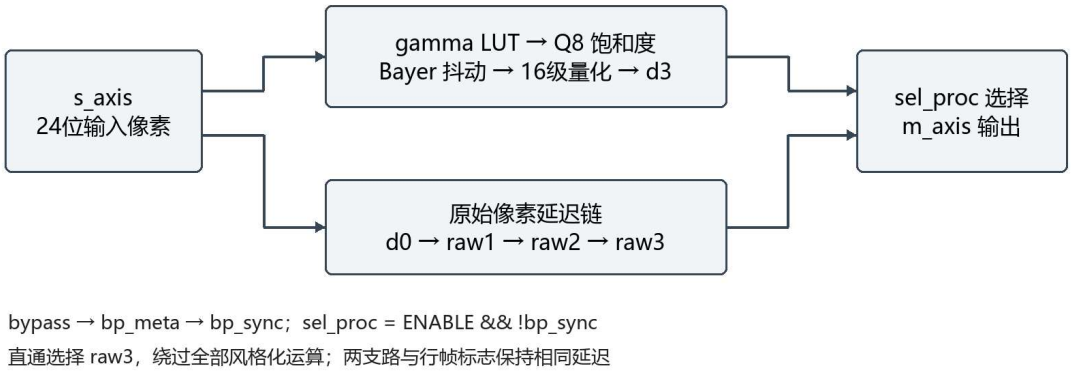


图 6　video_style_bypass 的点运算与等延迟原始支路

bypass 经 bp_meta、bp_sync 两级同步后控制 sel_proc。处理结果 d3 和原始数据 raw3 在输出端汇合；直通选 raw3，因此 Bayer 量化也被绕过。输出 m_axis 的像素与有效、行尾、帧首信号同步。显示域与采集域的模式响应存在同步和链路延迟，整机切换瞬态仍按测试记录说明。

风格化与模式切换

video_style_bypass 的默认参数为 GAMMA=45、SAT_Q8=256、POSTER_LEVELS=16。gamma 曲线由 gamma_lut.mem 的 256 项 8 位查找表决定；仅修改 GAMMA 注释参数不会重新生成曲线。饱和度公式为 clip(L+((SAT_Q8×(x−L))>>8))，当前系数 256 对应 1.0，不额外增强饱和度。

仅将灰度截到 16 的倍数，会把缓慢变化的暗部渐变压成可见台阶。最终 RTL 开启 DITHER_EN，以 8×8 Bayer 阈值 B∈[0,63] 生成 d=floor(B×16/64)，输出 min(x+d,255)&0xF0。随握手推进的坐标与像素同步，使量化误差分散到空间图案；高亮饱和区域仍可能有偏差，不能把特定灰度块的均值结果推广到任意画面。

风格化处理和原始数据支路均保留约四拍延迟，旁路选择不会单独破坏本模块的数据标志对齐。但输入侧两条支路总延迟不同，系统模式切换仍存在瞬态丢拍或画面跳动；模式开关仿真记录中 sent=13296、captured=13292，丢失 4 拍属于该次切换测试现象。

项目	最终实现
PL_KEY0	btns_tri_i[0]，L14；两级同步、20 ms 消抖。
按键极性	上电约 100 ms 识别空闲电平；复位后先松手等待，再按键。
模式 0	默认全处理，LED2 灭；处理输出写入帧缓存。
模式 1	直通，LED2 亮，mode_led 位于 L15；中值与 Kuwahara 继续运行，输出被丢弃。
跨时钟域	mode 从采集域送显示域，经两级同步；配合 ASYNC_REG 属性与 XDC 路径例外。

集成边界与时序状态

修改 HLS 几何时，应同步调整中值模板、Kuwahara 的 W/H、VDMA 和显示模式。约束中的模式信号属于准静态控制，路径例外仅适用于已同步的该控制通路。最终实现仍有建立时间违例，详见 3.3；生成比特流和板上正常出图并不等于全设计静态时序通过。

核心设计的工程取舍

从数值一致走向可持续的像素流

移植 Kuwahara 时，方差排序和均值取整会直接影响颜色输出。若四象限分数相同，比较顺序改变就可能选到另一组 RGB 均值；即使选区一致，统一采用“四舍五入”也会偏离参考的偶数舍入。例如通道和为 1608 时，均值 100.5 取 100；通道和为 1624 时，均值 101.5 取 102。这是舍入规则的算术示例，解释了为什么 RTL 同时检查余数和商的最低位。

本设计将方差比较化为整数判别式，省去浮点方差计算；16 个像素的通道均值则通过移位和舍入实现。数值等价性落实到明确的参数、平局和边界规则后，才获得内部 0 LSB 的对拍结果。这项实现特点适合固定半径、固定象限大小的流式核；改变半径时，位宽、窗口组织和均值分母均需重新设计。

把存储留给窗口，把等待交给流控

源码为 RGB 和 Q8 亮度各设置六级、深度 808 的行缓存，逻辑容量合计 6×808×(24+16)=193920 bit，约 23.67 KiB。作为量级参照，一帧 800×480 RGB888 需 1125 KiB，前者约为后者的 2.10%。这是存储位数推导，未计 BRAM 装填、寄存器和外围资源；整机仍保留 DDR 帧缓存，节省的是处理核的窗口存储需求。

边界处理同样有明确代价：扩展网格比 384000 个真实像素多 6264 个推进位置，约为有效像素数的 1.63%。无停顿时，每帧核心占用约 3.903 ms，而 40.05 fps 对应的帧周期约 24.969 ms。这说明当前数据率下计算预算有余量；上游气泡、下游反压和整机负时序裕量仍需单独考虑，不能据此承诺系统延迟。

窗口缓存的容量与排空拍数由真实视频几何共同约束。明确这些开销后，单像素流水的吞吐结论才有适用条件；整数规则和输出保持机制则分别保证参与计算及传输的数据符合预期。这样的实现便于在后续更换分辨率时，重新核算存储、排空和处理预算。

从异常画面到可核验的修正

约四重重影暴露了隐藏的行宽约束

工程记录中的早期上板画面出现约四重错位重影，窗口参数却来自看似合理的 VideoMode(640,480)。回查生成的 HLS 中值 RTL 后，团队发现其列循环固定为 800，应用层配置没有改变这个常量。仅按有效像素数计算，640 列相当于真实行宽的 0.8 倍，五段 640 像素才等于四段 800 像素。每行多出的 160 像素使缓存分行与真实行边界分离，窗口跨行取数由此发生水平错位。

修正将窗口核和 BD 参数统一为 W=800、H=480，并以 800×480 密集流重验，日志记录 captured=384000、mismatch=0、extra=0、flag_err=0；设计说明记载上板重影随之消失。640×480 显示仍裁去右侧 160 像素，这是当前显示屏显示范围的限制，与修复后的缓存行对齐是两个问题。

消隐期的一处差异决定采集替换是否兼容

自研采集模块起初在消隐期清零输出，而原实现保持最后一个像素。有效画面数据可能看起来相同，逐拍比较却会发现差异。团队据此修正保持行为，再比较数据、使能和同步等全部输出；历史测试覆盖启动、多帧、随机行场消隐及运行中复位，最终比较 91063 拍、0 失配。这个结果限定在测试激励范围内，提供了比“能显示图像”更具体的接口兼容证据。

量化色带需要可重复的小样例

在暗部渐变中，16 级截断会使数值 167 统一落到 160。DITHER_FIX.md 保存的测试采用 8×8 恒值灰度块，gamma(100)=167；加入 Bayer 后，64 个输出均在合法量化级，块均值回到 167。该例说明抖动如何通过空间分配缓解台阶，也划定了结论范围：它验证一个固定输入块，不能替代最终显示链在任意画面、坐标和反压条件下的完整自校验。

三个实例分别追问几何、接口和显示误差的来源。修正是否有效，由对应的帧计数、逐拍日志或固定样例回答；第 3.3 节汇总的各类证据也沿用这一划分。

2.3 软件系统介绍

2.3.1 软件整体介绍

PS 端使用包含 pynq.lib.ov5640 的厂商 PYNQ 镜像 Release 2025_08_27。主程序 src/ps/kuwahara_ps.py 与同名 Notebook 提供运行入口，离线参考和验证使用 NumPy、Pillow；实时算法主体不依赖 PC 或云端。



图 7 PS 主控制程序流程

2.3.2 软件各模块介绍

以下按入口、业务调用及底层运算展开四个源码模块的 35 个函数，给出流程及关键变量。库函数以调用点说明，条件和循环在流程框内标明。

PS 主控制程序 src/ps/kuwahara_ps.py

main 函数

输入：命令行 args.bit / args.frames。输出：PNG 文件、终端状态；无返回值。

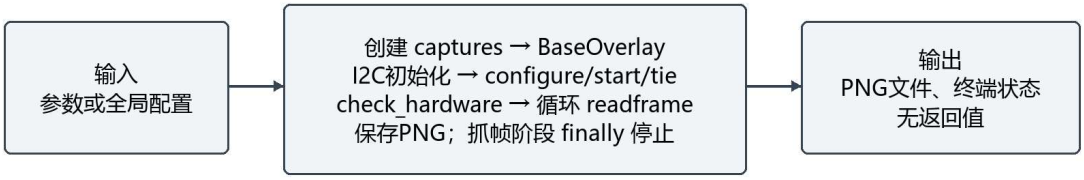


图 8 kuwahara_ps.py 中 main 的处理流程

check_hardware 函数

输入：hwh_path=HWH_PATH。输出：打印 OK / MISSING；无返回值。

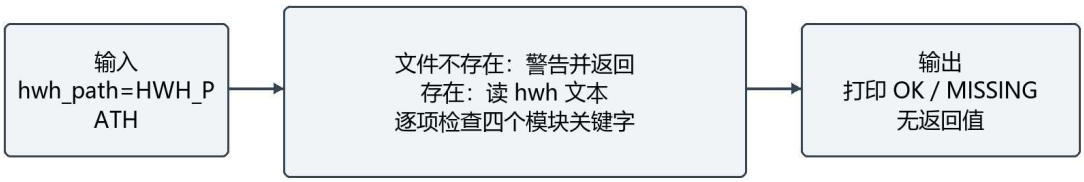


图 9 kuwahara_ps.py 中 check_hardware 的处理流程

main 仅对抓帧阶段执行 finally 清理。check_hardware 只检查 hwh 文本；默认 ./base.hwh 应与加载的 bit 配套。

板端测量与诊断　board/measure.py

main 函数

输入：cmd / seconds / interval / bit / outdir。输出：终端报告；compare 另存图片。

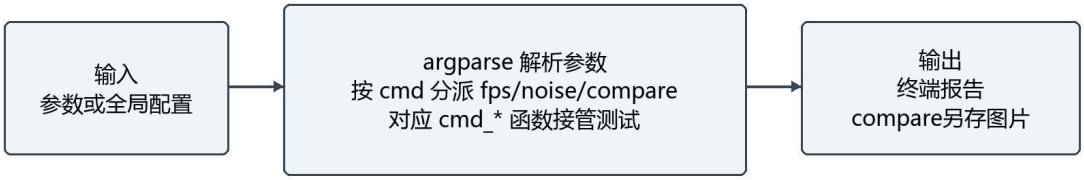


图 10　measure.py 中 main 的处理流程

cmd_fps 函数

输入：args.bit / args.seconds。输出：elapsed / reads / unique / fps。



图 11　measure.py 中 cmd_fps 的处理流程

cmd_noise 函数

输入：args.bit / args.interval。输出：两模式帧均值与帧间差异。

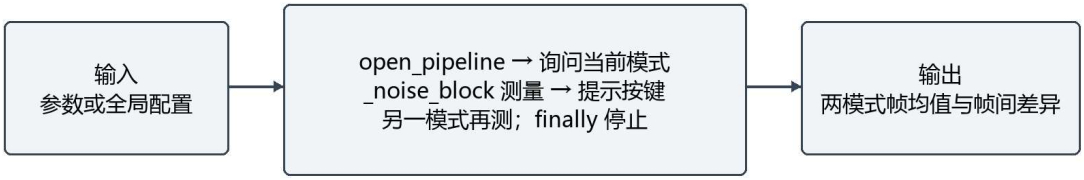


图 12　measure.py 中 cmd_noise 的处理流程

cmd_compare 函数

输入：args.bit / args.outdir。输出：cmp_*.png；诊断统计，非验收。

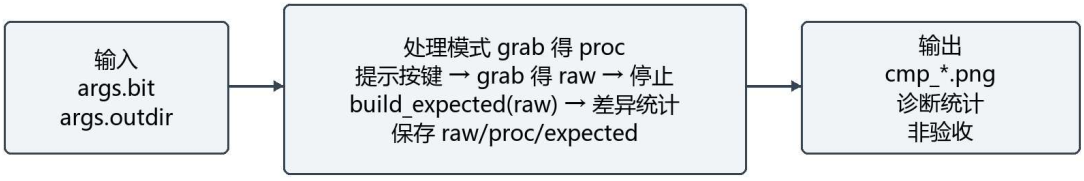


图 13　measure.py 中 cmd_compare 的处理流程

板端测量与诊断　续　board/measure.py

open_pipeline 函数

输入：bit_path / freeze / wb。输出：base, cam, hdmi 三个对象。



图 14　measure.py 中 open_pipeline 的处理流程

measure_fps 函数

输入：cam / seconds=10.0。输出：含 elapsed_s/reads/unique_frames/fps 的字典。



图 15　measure.py 中 measure_fps 的处理流程

_noise_block 函数

输入：cam / label / n=4 / interval。输出：frames 列表；终端统计。

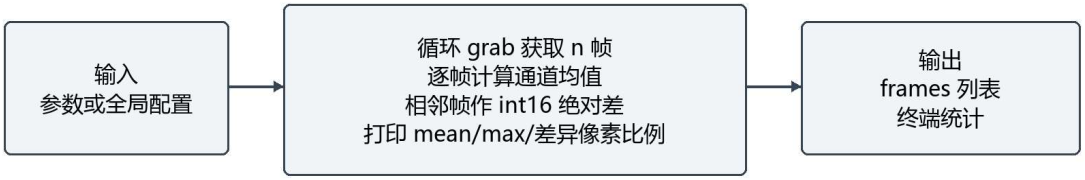


图 16　measure.py 中 _noise_block 的处理流程

grab 函数

输入：cam / settle=0.2。输出：连续布局的像素数组。



图 17　measure.py 中 grab 的处理流程

板端测量与诊断　续　board/measure.py

build_expected 函数

输入：raw：H×W×3 图像。输出：H×W×3 int16 期望图。

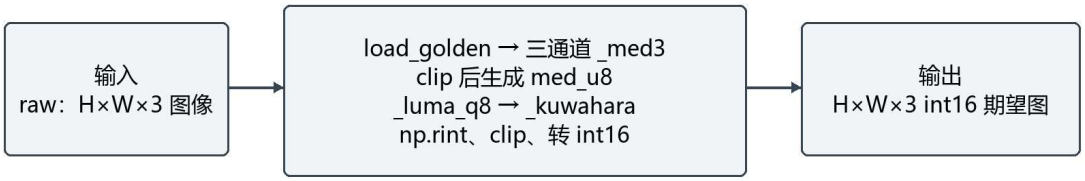


图 18　measure.py 中 build_expected 的处理流程

load_golden 函数

输入：path=GOLDEN。输出：参考模块对象 ref。



图 19　measure.py 中 load_golden 的处理流程

measure_fps 实际使用 SHA1 对像素字节摘要去重，fps 为唯一图像数量除以耗时；静态无噪声场景可能低估帧更新率。open_pipeline 的 freeze、wb 参数目前仅被传入，函数体未实现对应寄存器设置，不应声称已经冻结曝光或白平衡。grab 使用 ascontiguousarray 保证连续布局，但不保证创建独立副本。

离线参考算法　data/style_cartoon_640x480.py

run 函数

输入：in_path / out_path / cfg。输出：输出文件与 out 数组。

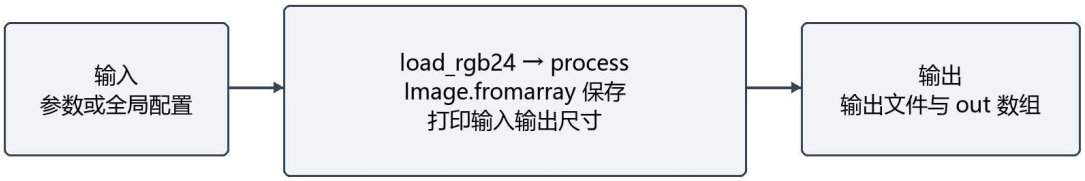


图 20　style_cartoon_640x480.py 中 run 的处理流程

process 函数

输入：rgb_u8 / cfg / posterize。输出：H×W×3 uint8 结果。

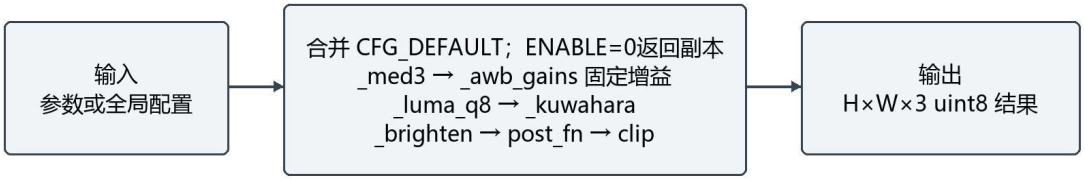


图 21　style_cartoon_640x480.py 中 process 的处理流程

load_rgb24 函数

输入：path。输出：(480,640,3) uint8 数组。



图 22　style_cartoon_640x480.py 中 load_rgb24 的处理流程

selftest 函数

输入：in_path。输出：打印比较与耗时；无返回值。

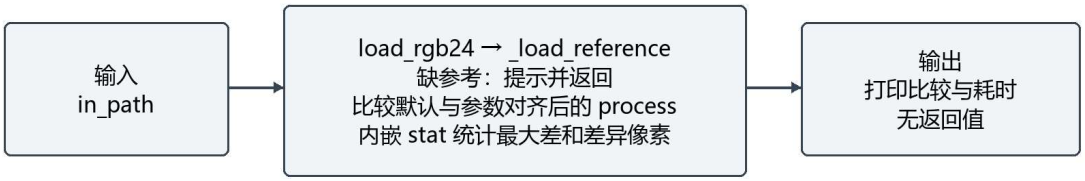


图 23　style_cartoon_640x480.py 中 selftest 的处理流程

离线参考算法　续　data/style_cartoon_640x480.py

_med3 函数

输入：x：单通道二维数组。输出：与 x 同尺寸的中值图。

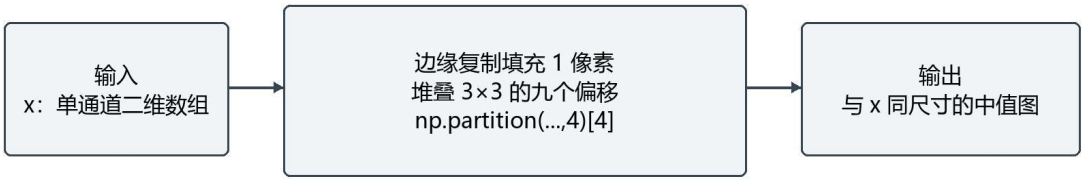


图 24　style_cartoon_640x480.py 中 _med3 的处理流程

_luma_q8 函数

输入：rgb：H×W×3。输出：H×W 的 int64 Q8 亮度。



图 25　style_cartoon_640x480.py 中 _luma_q8 的处理流程

_awb_gains 函数

输入：cfg 的 AWB_GAIN_R/G/B。输出：长度 3 的 gains 数组。

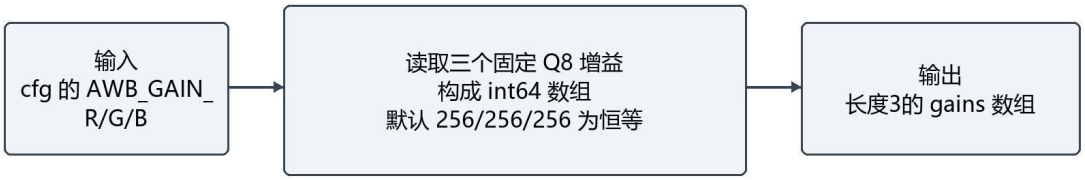


图 26　style_cartoon_640x480.py 中 _awb_gains 的处理流程

_kuwahara 函数

输入：rgb / r / luma_y。输出：H×W×3 int64 保边平滑图。



图 27　style_cartoon_640x480.py 中 _kuwahara 的处理流程

离线参考算法　续　data/style_cartoon_640x480.py

_brighten 函数

输入：rgb / cfg。输出：int64 颜色增强图。

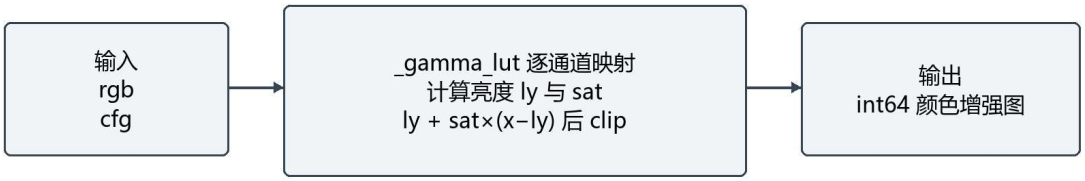


图 28　style_cartoon_640x480.py 中 _brighten 的处理流程

_gamma_lut 函数

输入：g100。输出：256 项 int64 查找表。



图 29　style_cartoon_640x480.py 中 _gamma_lut 的处理流程

_build_gamma_lut 函数

输入：g100。输出：256 项 int64 gamma 表。

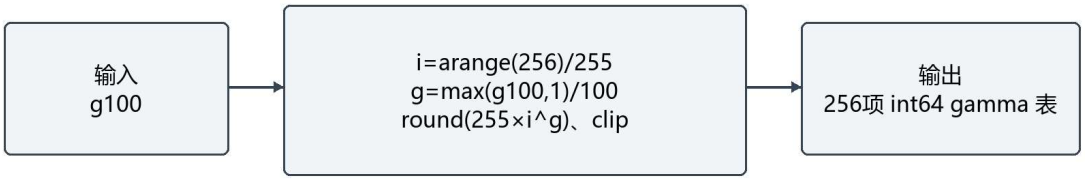


图 30　style_cartoon_640x480.py 中 _build_gamma_lut 的处理流程

_posterize 函数

输入：x / levels。输出：int64 截断量化图。

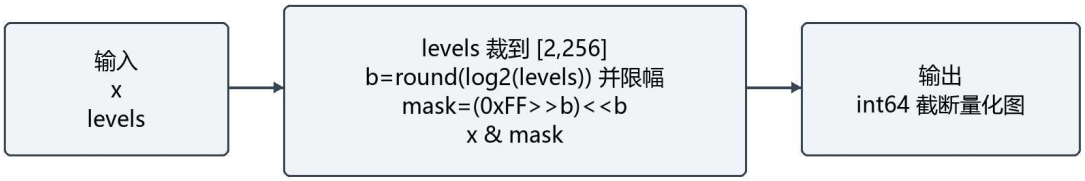


图 31　style_cartoon_640x480.py 中 _posterize 的处理流程

离线参考算法　续　data/style_cartoon_640x480.py

_posterize_legacy 函数

输入：x / levels。输出：int64 旧量化结果，仅回归使用。

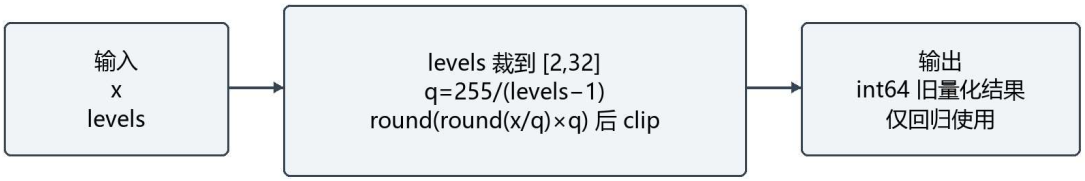


图 32　style_cartoon_640x480.py 中 _posterize_legacy 的处理流程

_load_reference 函数

输入：同目录 style_reference.py 路径。输出：参考模块或 None。

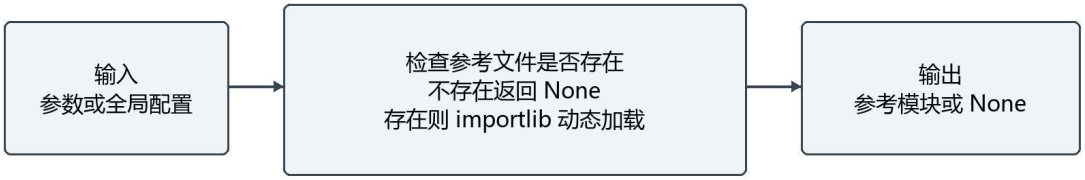


图 33　style_cartoon_640x480.py 中 _load_reference 的处理流程

默认 process 采用固定 Q8 白平衡增益和截断量化，参考函数 _posterize 不含最终显示 RTL 的 Bayer 分支。_posterize_legacy、_load_reference 与 selftest 用于可选历史回归；缺少 style_reference.py 时不形成有效比较结果。

整数行为模型与向量生成　sim/verify_kuwahara.py

compare_full 函数

输入：REF_PATH / IMG_PATH / W_FULL / H_FULL。输出：内部与复制边界均无差异则 True。



图 34　verify_kuwahara.py 中 compare_full 的处理流程

load_reference 函数

输入：全局 REF_PATH。输出：参考模块或 None。



图 35　verify_kuwahara.py 中 load_reference 的处理流程

load_image 函数

输入：path / w / h / y0 / x0。输出：h×w×3 uint8 图像。



图 36　verify_kuwahara.py 中 load_image 的处理流程

gen_tb_vectors 函数

输入：IMG_PATH / TB_W / TB_H。输出：小图 HEX 路径 p_in,p_exp。



图 37　verify_kuwahara.py 中 gen_tb_vectors 的处理流程

整数行为模型与向量生成　续　sim/verify_kuwahara.py

gen_tb_full 函数

输入：IMG_PATH / FULL_W=800 / FULL_H=480。输出：全尺寸 HEX 路径 p_in,p_exp。



图 38　verify_kuwahara.py 中 gen_tb_full 的处理流程

kuwahara_rtl 函数

输入：rgb_u8 / r=3 / edge。输出：H×W×3 int64 整数模型结果。

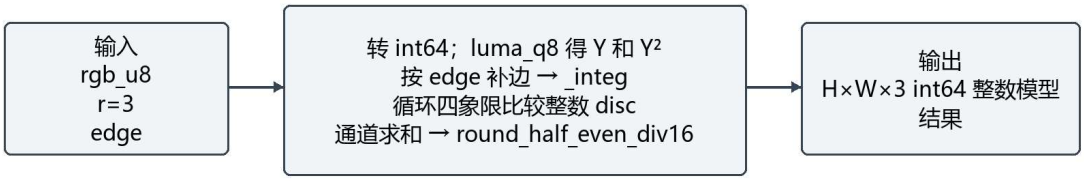


图 39　verify_kuwahara.py 中 kuwahara_rtl 的处理流程

luma_q8 函数

输入：rgb：H×W×3。输出：H×W 的 Q8 亮度 Y。



图 40　verify_kuwahara.py 中 luma_q8 的处理流程

_integ 函数

输入：x：二维数组。输出：int64 积分图 ii。

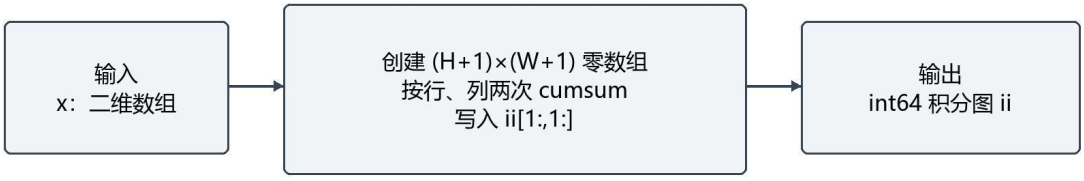


图 41　verify_kuwahara.py 中 _integ 的处理流程

整数行为模型与向量生成　续　sim/verify_kuwahara.py

round_half_even_div16 函数

输入：s：整数通道和。输出：int64 偶数舍入均值。

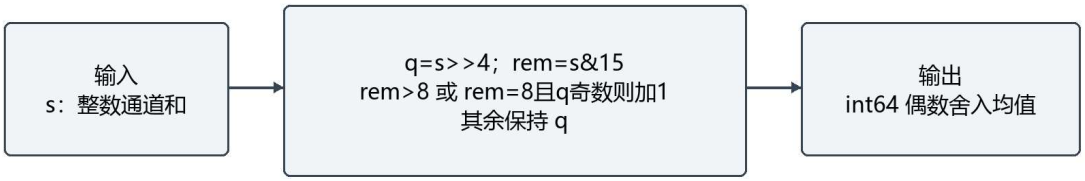


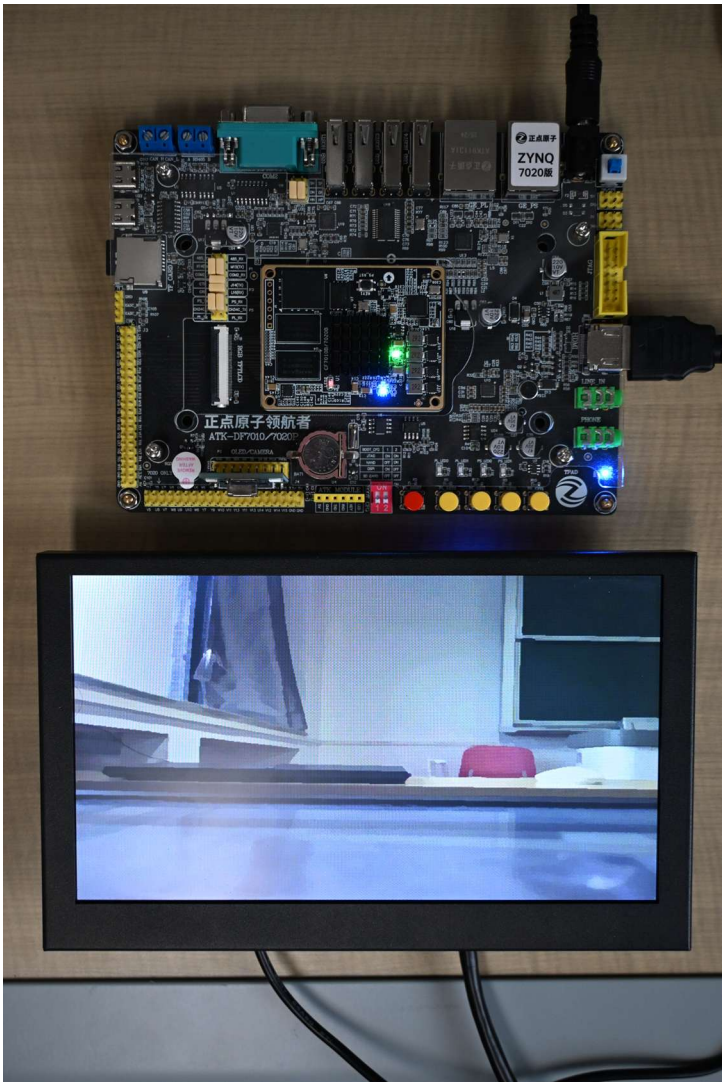
图 42　verify_kuwahara.py 中 round_half_even_div16 的处理流程

模块入口根据 --tb / --tb-full 调用向量生成。脚本的参考模型、测试图与输出目录已统一为本包内相对路径（../data/ 与 sim/），无需再做路径映射；2026-11-04 复跑结果为内部像素 300516/300516 逐位一致。整数模型补边可选 zero/replicate，端到端比较时须与目标边界规则一致。

第三部分　完成情况及性能参数

3.1 整体介绍

系统由连接开发板的 OV5640 摄像头、Zynq 7020 开发板和 HDMI 显示设备构成。下图为运行现场实拍，显示器与开发板同框；后续照片给出板卡的俯视、斜视和摄像头连接细节。现场照片展示实物组成及出图状态，数值一致性和帧率仍以仿真、测量记录说明。



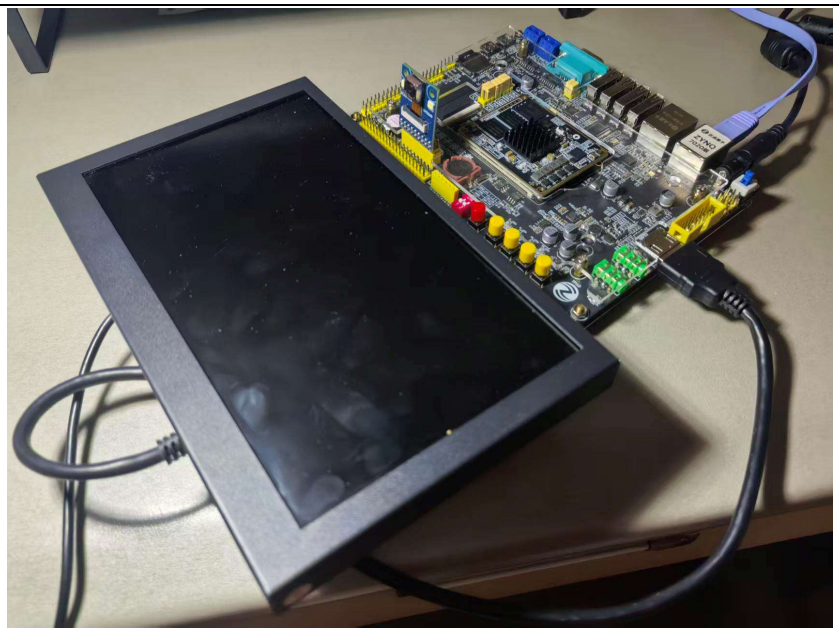


图 43　系统运行现场实拍　显示设备与开发板同框

3.2 工程成果

3.2.1 机械成果

系统采用开发板原有底板、核心板及透明保护板，通过支撑柱固定；摄像头插接于扩展接口。本作品未设计独立机械传动或定制外壳，结构成果体现在板卡安装与外设连接，实物见本节照片。

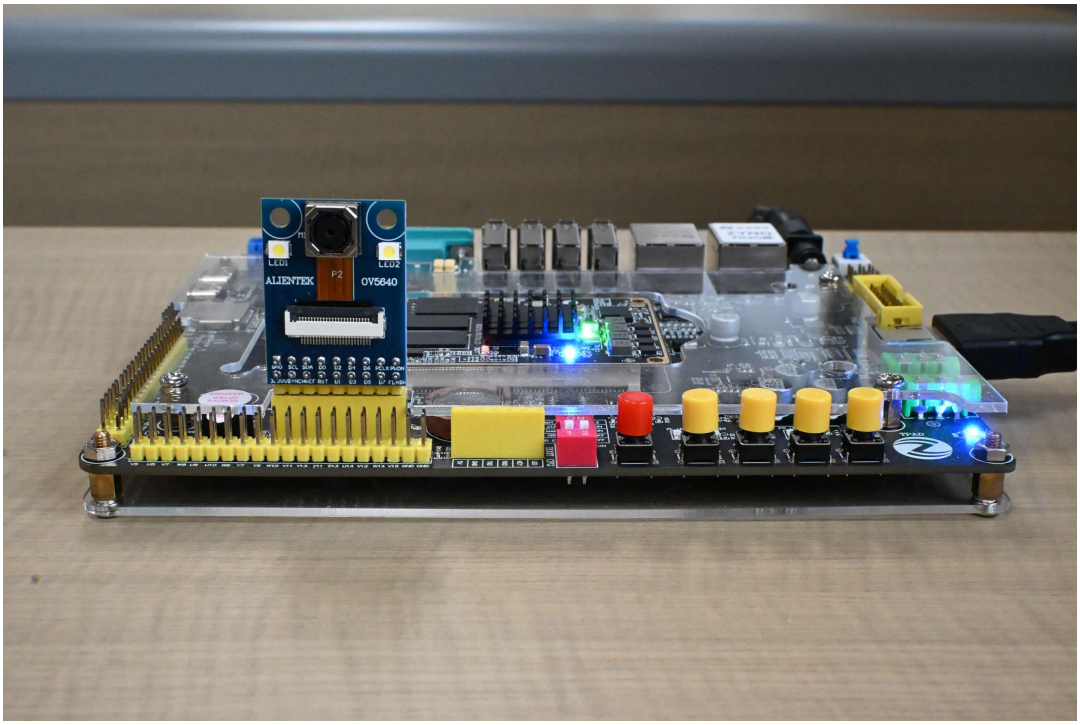


图 44　板卡正面及安装结构实拍　OV5640、保护板和支撑柱

摄像头模块标识为 OV5640，插接位置对应底板 OLED/CAMERA 接口。正面可见摄像头、按键和透明保护板；板级连线及数字接口关系在前述电路设计中说明。

3.2.2 电路成果

硬件成果为基于现有开发板实现的可加载设计，交付 src/base.xpr、base.srcs、ip_repo_local 及配套 bit/hwh。实物照片展示载板和核心板、已连接的 HDMI 线、散热器及摄像头模块；未连接的以太网、USB 和串口等外围不作为本作品图像通路功能进行宣称。

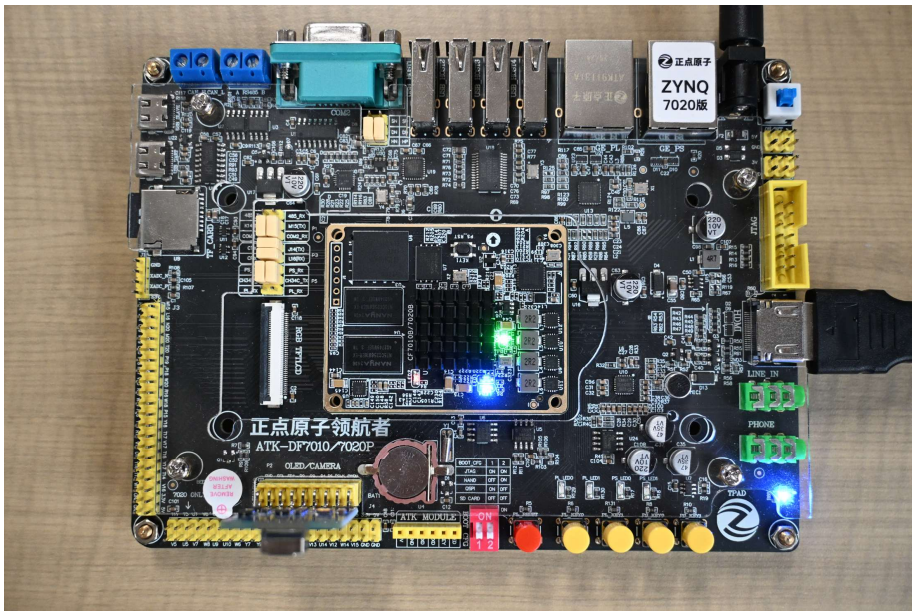


图 45　板卡俯视实拍　核心板及外围接口

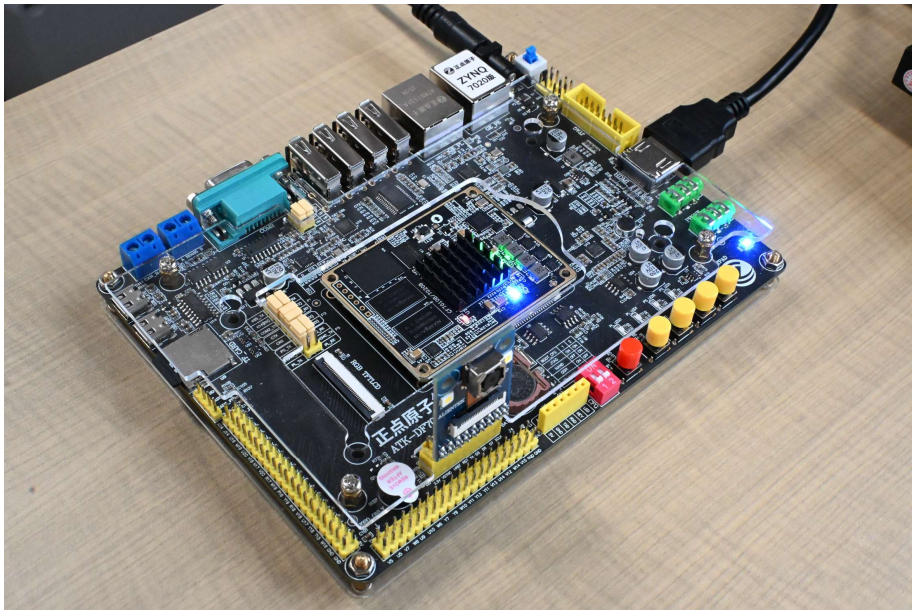


图 46　板卡斜视实拍　摄像头模块与 HDMI 连接

3.2.3 软件成果

板上控制软件完成摄像头初始化、通路启动和抓帧；现场显示状态见前述系统实拍。下图及后续视频截帧来自离线模型和电脑算法处理，用于说明视觉效果。实物照片、离线画面和功能仿真分别提供不同层面的证据。



图 47　离线参考效果拼图　非板上实拍

图 47 上排依次为输入、中值结果和 Kuwahara 整数模型；下排为浮点参考、风格化参考链和放大的边界差异。平坦区域形成连续色块，边缘仍可辨识。图中的历史参考链不能替代最终 Bayer 显示 RTL 的逐像素验证。

电脑算法视频帧对照

从原视频 DSC_3384.MOV 与电脑算法处理视频 cartoon_int_1080p60.mp4 中选取同一帧序号进行对照。素材为 1920×1080、约 59.94 fps；以下均为电脑处理效果，非板上实拍，不代表 FPGA 的输出分辨率或实测帧率。

左列　原视频　　右列　电脑算法处理后

图 48　约 2.50 s　建筑与树叶交界局部对照

两图采用相同裁剪范围，观察建筑轮廓与叶片纹理的变化

图 49　约 5.51 s　建筑与树木整体对照

处理后亮度提高，细碎叶片纹理呈现较平滑的色块

图 50　约 8.51 s　树冠层次对照

处理后形成柔和的绘画色调，树冠外轮廓仍可辨识

3.3 特性成果

功能正确性与证据范围

测试项目	结果	判据与范围
全尺寸 Kuwahara RTL	384000/384000 mismatch=0 extra=0，flag_err=0	800×480，GAP=0；与零边界整数期望比较，同时检查数量与标志。
小尺寸流控回归	gap0 / gap12 日志 PASS	对应 testbench 条件；覆盖密集流和有间隔输入。
采集 IP 逐拍等价	91063 拍 0 失配，PASS	并排例化原实现和自研模块，包含启动、消隐及运行中复位等激励。
模式开关	13292 输出 0 失配，PASS	假处理链测试；输入 13296 拍，切换瞬态少 4 拍，不证明无缝切换。
整数模型与 golden	内部 0/300516 差异 最大差 0 LSB	640×480 测试图，经中值后，去除最外三行三列。
边界策略对照	3157/307200 差异 最大差 165 LSB	零填充与边缘复制差异；统一为复制边界后，全图差异为 0。

为什么既报告“384000 像素无失配”，又保留边界差异？两个测试回答的问题不同。RTL 仿真核对的是零边界整数期望，离线模型对拍则以边缘复制的浮点参考为基准。640×480 测试图的内部 300516 像素完全相同；全图差异集中在三像素边界，统一边界策略后消失。模型结果于 2026 年 11 月 3 日复核，RTL 结果来自 sim/logs/ 保存的历史记录。

DITHER_FIX.md 记录了 8×8 恒值测试：gamma(100)=167，输出 64 像素，块均值为 167，level_errors=0。该记录说明此测试块的抖动量化行为；现有 verify_style.py 的旧模型未覆盖最终 Bayer 坐标分支，不能据此宣称最终显示模块已完成全图逐位对拍。

板端不同时间抓帧受场景变化、曝光和输入扰动影响，Kuwahara 的离散选区也可能改变，因此异帧差异仅用于诊断。对固定数字输入的数值正确性，应以对应模型和 RTL 自校验为依据。

板上性能与理论处理能力

已有测量记录的条件为静止场景、OV5640 RGB565 输入 800×480、HTS=1800、VTS=1000，HDMI 配置 640×480。measure.py fps --seconds 10 的记录为约 10 s 内 401 个非重复读取帧、40.05 fps。该结果反映采集帧读取表现，不等同于用仪器测得的 HDMI 刷新率。

指标	结果与计算口径
输入有效像素需求	800×480×40.05≈15.38 Mpixel/s。
处理核峰值	1 pixel/clk×100 MHz=100 Mpixel/s，约为有效输入需求的 6.50 倍；不含 flush 开销。
每帧扩展网格	483×808=390264 拍；连续推进时约 3.903 ms，占 40.05 fps 帧周期的约 15.63%。
核心窗口延迟	3×808+3+5=2432 拍；连续推进时 24.32 μs，输入气泡或反压会增加实际等待。
相邻帧差异记录	mean 15.8～21.9 LSB，差异像素约 99%；照明及自动曝光影响属于诊断解释。
处理直通对照	单次平均差 5.97 LSB，仅说明两次抓帧不同，不能单独证明算法等价。

实现资源与静态时序

资源	使用量	器件占比
Slice LUTs	22864 / 53200	42.98%
Slice Registers	27514 / 106400	25.86%
Block RAM Tile	22.5 / 140	16.07%
DSP48E1	19 / 220	8.64%

数据取自最终 Routed 状态 impl_utilization.rpt。impl_timing_summary.rpt 中 100 MHz 域 WNS=−0.571 ns、TNS=−23.985 ns、失败端点 107；142.857 MHz 域 WNS=−0.115 ns、TNS=−0.173 ns、失败端点 3。工程尚未整体时序收敛。历史 kw_timing.rpt 的所列核心路径裕量为 +0.318 ns，来自较早构建。

第四部分　总结

4.1 可扩展之处

后续应优先修复最终实现中 HLS 与外围路径的时序违例，重新建立可重复的板级测量记录，并将采集、中值、窗口处理及显示统一到支持的分辨率。可增加帧边界模式切换、运行期参数寄存器和最终 Bayer 链路的完整自校验。现有行缓存已经使用环形 BRAM，进一步优化应围绕存储组织、位宽和计算资源复用展开。

4.2 心得体会

项目推进中，一个反复出现的问题是：单个模块的结果正确，为什么整条视频链仍会花屏？行宽失配给出了具体答案。窗口核曾按显示模式设置为 640 列，而中值 IP 实际按 800 列输出，缓存由此把相邻行的像素混在一起。把真实流几何作为共同约束，比继续调整滤波参数更接近问题根源。

数值与接口也需要不同的验收方法。偶数舍入用算术样例和参考对拍核对，帧末排空用输出计数与行帧标志核对；采集模块替换，则连消隐期的数据保持都要逐拍检查。已保存的 91063 拍等价性记录说明这种细查的价值。相同原则也用于性能表述：100 Mpixel/s 是核心理论峰值，40.05 fps 是既有采集读取记录，两者不能替代端到端延迟测量。

大模型参与推导、实现和日志分析，人工提供板级现象并决定方案是否保留。有效的协作发生在可核验的反馈中，例如用“约四重重影”追查行宽，用消隐期失配修正采集输出，而不是仅凭画面观感认定正确。阶段记录归档于 report/LLM_COLLAB_LOG.md，七项经验保存在 skill/README.md。后续复用仍需重新核对几何、时序和测试条件；自研与第三方组件的来源及许可也应随代码保留。

第五部分　参考文献

[1] KUWAHARA M, HACHIMURA K, EIHO S, et al. Processing of RI-Angiocardiographic Images[M]//PRESTON K, ONOE M. Digital Processing of Biomedical Images. Boston, MA: Springer, 1976: 187-202. DOI: 10.1007/978-1-4684-0769-3_13.

[2] AMD. AXI4-Stream Video IP and System Design Guide: UG934[EB/OL]. [2026-11-03]. https://docs.amd.com/r/2025.1-English/ug934_axi_videoIP.

[3] AMD. Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator: UG994[EB/OL]. [2026-11-03]. https://docs.amd.com/r/2025.1-English/ug994-vivado-ip-subsystems.